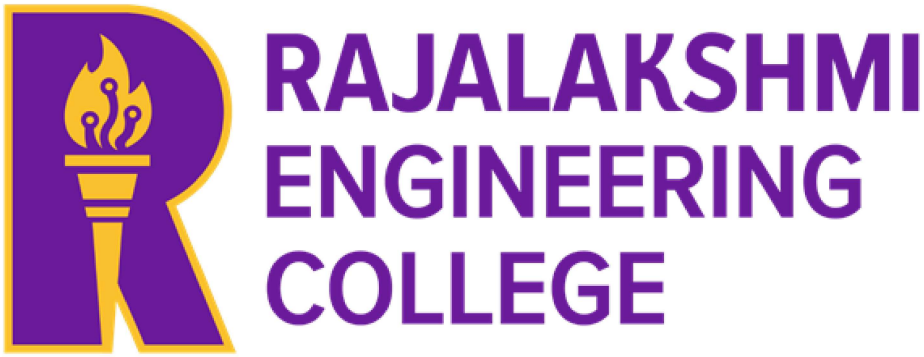
RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602105



AI23331

FUNDAMENTALS OF MACHINE LEARNING LABORATORY

Laboratory Observation Note Book

NAME : MOHAMED ZAIDH A

YEAR/BRANCH/SEC : 3RD YEAR, COMPUTER SCIENCE & DESIGN, A

REGISTER NO. : 231701032

SEMESTER: VI th SEMESTER

ACADEMIC YEAR : 2025-2026

BONAFIDE CERTIFICATE

Certified that this laboratory manual is the bonafide work of **" MOHAMED ZAIDH A (231701036) "** who carried out the lab work for the subject AI23331- Fundamentals of machine learning.

SIGNATURE

Mrs. Kalpana D, M.E.,

Assistant Professor(SG)

Supervisor

Computer Science and Design

Rajalakshmi Engineering College

Chennai – 602105

Submitted to Project and Viva Voce Examination for the subject **AI23331- Fundamentals of machine learning** held on_____.

Internal Examiner External Examiner

INDEX PAGE

S.No.	Experiment title	Date	Sign
1	A python program to implement univariate regression, bivariate regression and multivariate regression.		
2	A python program to implement Simple linear regression using Least Square Method		
3	A python program to implement logistic model		
4	A python program to implement single layer perceptron.		
5	A python program to implement multi layer perceptron with back propagation		
6	A python program to do face recognition using SVM classifier		
7	A python program to implement decision tree.		
8	A python program to implement boosting.		
9	A python program to implement KNN and K-means.		
10	A python program to implement dimensionality reduction – PCA		

EXPERIMENT: 1

Python program for univariate, bivariate, and multivariate regression

Aim:

To write a python program for univariate, bivariate, and multivariate regression

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
url = 'https://raw.githubusercontent.com/MohamedZaidhA/Machine
LEarning/21eb44c703b53ce7014cd008e67fe34d33d80caf/Iris.csv'
df = pd.read_csv(url)
display(df.head())
display(df.shape)


#univariate for sepal width
df_Iris_setosa=df.loc[df['Species']=='Iris-setosa']
df_Virginica=df.loc[df['Species']=='Iris-virginica']
df_Versicolor=df.loc[df['Species']=='Iris-versicolor']
plt.scatter(df_Iris_setosa['SepalWidthCm'],np.zeros_like(df_Iris_setosa['SepalWidthCm']))
plt.scatter(df_Virginica['SepalWidthCm'],np.zeros_like(df_Virginica['SepalWidthCm']))
plt.scatter(df_Versicolor['SepalWidthCm'],np.zeros_like(df_Versicolor['SepalWidthCm']))
plt.xlabel('SepalWidthCm')
```

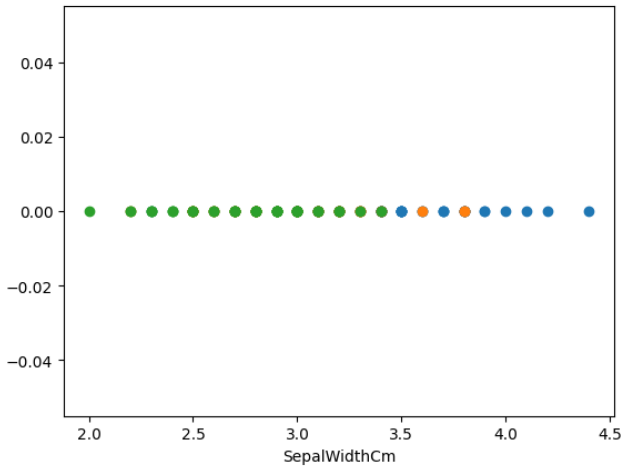
```
plt.show()


#univariate for sepal length
df_Iris_setosa=df.loc[df['Species']=='Iris-setosa']
df_Virginica=df.loc[df['Species']=='Iris-virginica']
df_Versicolor=df.loc[df['Species']=='Iris-versicolor']
plt.scatter(df_Iris_setosa['SepalLengthCm'],np.zeros_like(df_Iris_setosa['SepalLengthCm']))
plt.scatter(df_Virginica['SepalLengthCm'],np.zeros_like(df_Virginica['SepalLengthCm']))
plt.scatter(df_Versicolor['SepalLengthCm'],np.zeros_like(df_Versicolor['SepalLengthCm']))
plt.xlabel('SepalLengthCm')
plt.show()

#univariate for petal width
df_Setosa=df.loc[df['Species']=='Iris-setosa']
df_Virginica=df.loc[df['Species']=='Iris-virginica']
df_Versicolor=df.loc[df['Species']=='Iris-versicolor']
plt.scatter(df_Setosa['PetalWidthCm'],np.zeros_like(df_Setosa['PetalWidthCm']))
plt.scatter(df_Virginica['PetalWidthCm'],np.zeros_like(df_Virginica['PetalWidthCm']))
plt.scatter(df_Versicolor['PetalWidthCm'],np.zeros_like(df_Versicolor['PetalWidthCm']))
plt.xlabel('PetalWidthCm')
plt.show()

#univariate for petal length
df_Setosa=df.loc[df['Species']=='Iris-setosa']
```

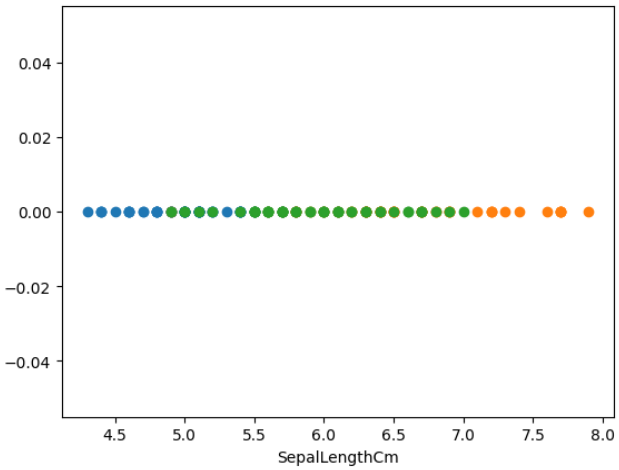
```
df_Virginica=df.loc[df['Species']=='Iris-virginica']
df_Versicolor=df.loc[df['Species']=='Iris-versicolor']
plt.scatter(df_Setosa['PetalLengthCm'],np.zeros_like(df_Setosa['PetalLengthCm']))
plt.scatter(df_Virginica['PetalLengthCm'],np.zeros_like(df_Virginica['PetalLengthCm']))
plt.scatter(df_Versicolor['PetalLengthCm'],np.zeros_like(df_Versicolor['PetalLengthCm']))
plt.xlabel('PetalLengthCm')
plt.show()
#bivariate sepal.width vs petal.width
sns.FacetGrid(df,hue='Species',height=5).map(plt.scatter,"SepalWidthCm","PetalWidthCm").ad
d_legend();
#bivariate sepal.length vs petal.length
sns.FacetGrid(df,hue='Species',height=5).map(plt.scatter,"SepalLengthCm","PetalLengthCm").a
dd_legend();
plt.show()
#multivariate all the features
sns.pairplot(df,hue="Species",height=2)
```

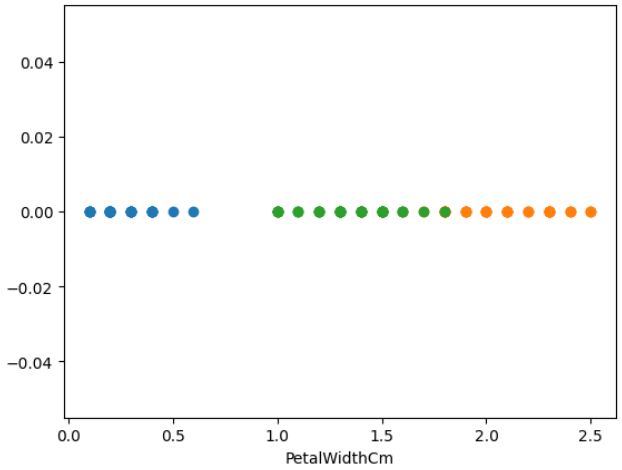
OUTPUT:

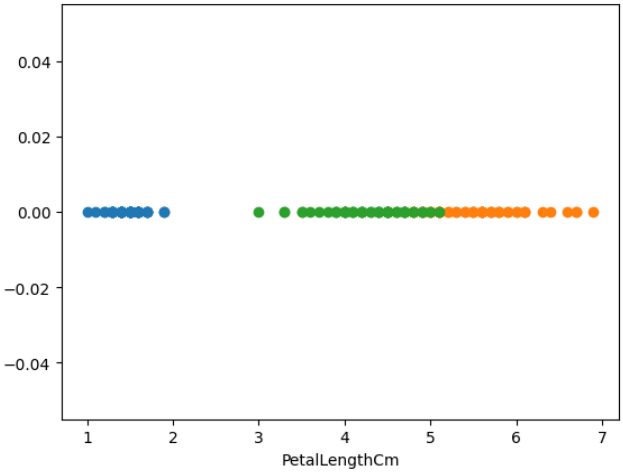
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa
2	3	4.7	3.2	1.3	0.2	Iris-setosa
3	4	4.6	3.1	1.5	0.2	Iris-setosa
4	5	5.0	3.6	1.4	0.2	Iris-setosa

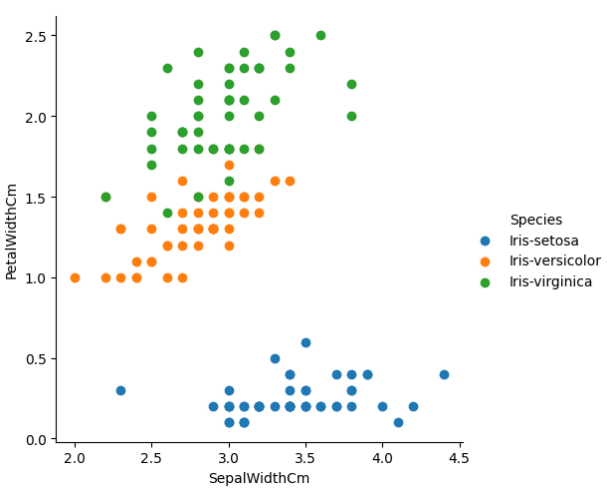
```
(150, 6)
```

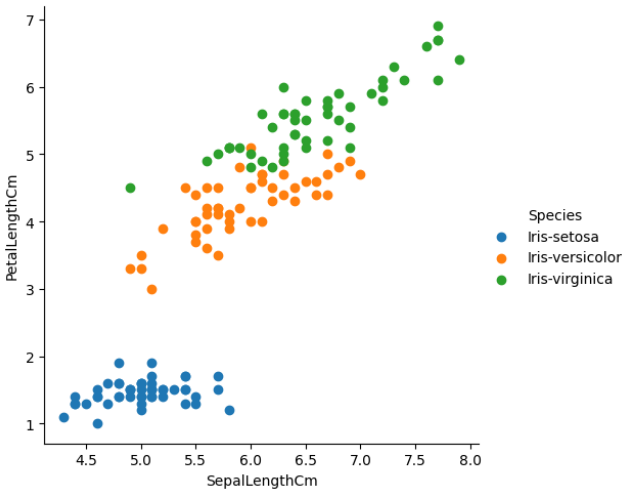

0.04
0.02
0.00
−0.02
−0.04
2.0
2.5
3.0
3.5
4.0
4.5
SepalWidthCm

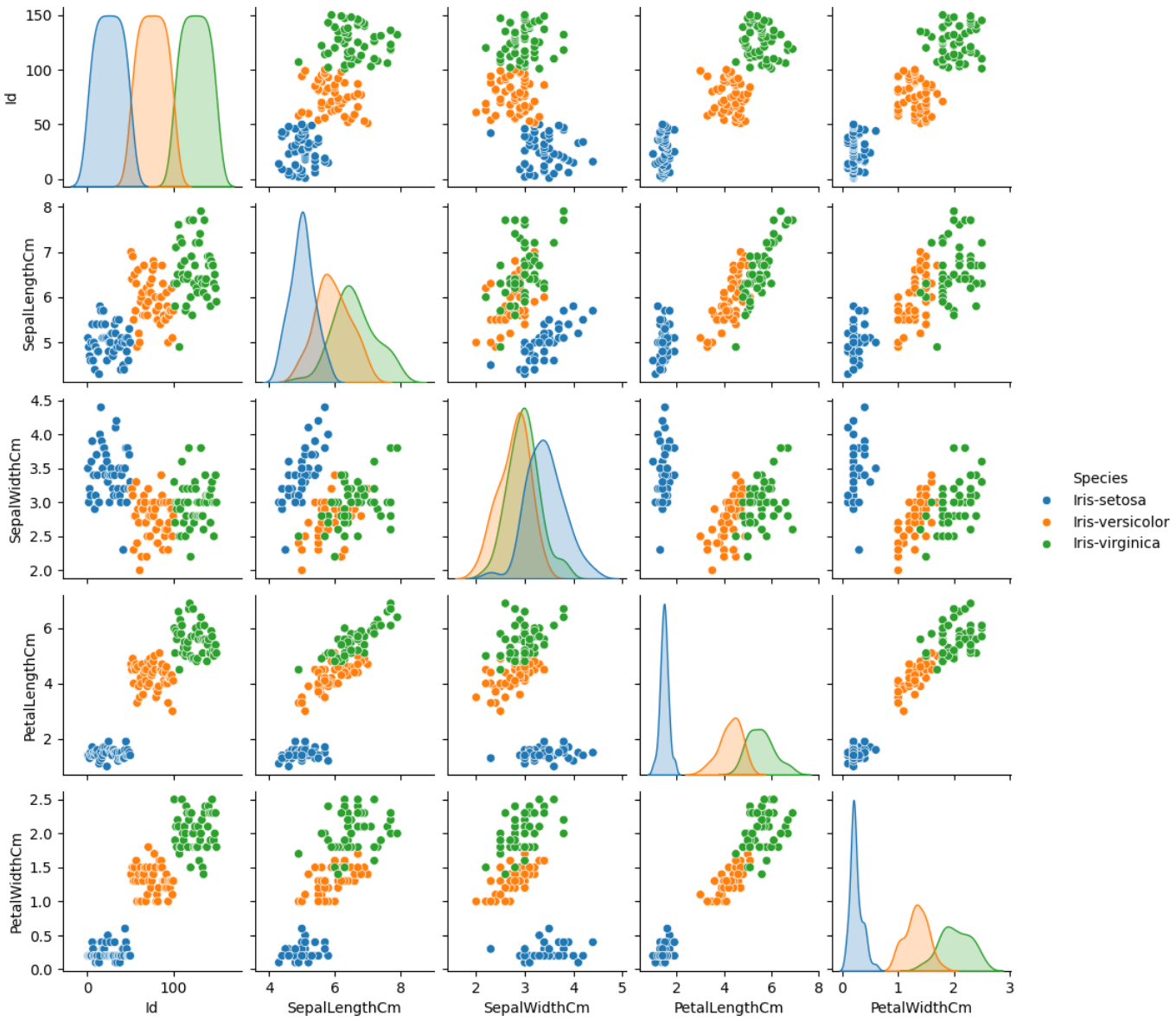

0.04
0.02
0.00
−0.02
−0.04
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
SepalLengthCm


0.04
0.02
0.00
−0.02
−0.04
0.0
0.5
1.0
1.5
2.0
2.5
PetalWidthCm


0.04
0.02
0.00
−0.02
−0.04
1
2
3
4
5
6
7
PetalLengthCm


2.5
2.0
1.5
1.0
0.5
0.0
2.0
2.5
3.0
3.5
4.0
4.5
PetalWidthCm
SepalWidthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica


7
6
5
4
3
2
1
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0
PetalLengthCm
SepalLengthCm
Species
Iris-setosa
Iris-versicolor
Iris-virginica



RESULT:

Python program for univariate, bivariate, and multivariate regression was implemented successfully.

EXPERIMENT: 2

SIMPLE LINEAR REGRESSION USING LEAST SQUARE METHOD

Aim:

To implement Simple Linear Regression using the Least Squares Method in Python to analyze the relationship between Playing Hours (independent variable) and Performance Score (dependent variable).

Procedure:

1. Create a CSV dataset named Salary_dataset.csv containing:
    - Years of Experience (independent variable)
    - Salary (dependent variable)
2. Import the required Python libraries:
    - pandas (for data handling)
    - numpy (for mathematical calculations)
    - matplotlib (for visualization)
3. Load the dataset using pandas.
4. Extract independent variable (x) and dependent variable (y).
5. Calculate the mean of x and y.
6. Compute the slope (b_1) using the formula:

$$b_1 = \frac{\sum(x - \bar{x})(y - \bar{y})}{\sum(x - \bar{x})^2}$$

7. Compute the intercept (b_0) using:

$$b_0 = \bar{y} - b_1\bar{x}$$

8. Form the regression equation:

$$y = b_0 + b_1 x$$

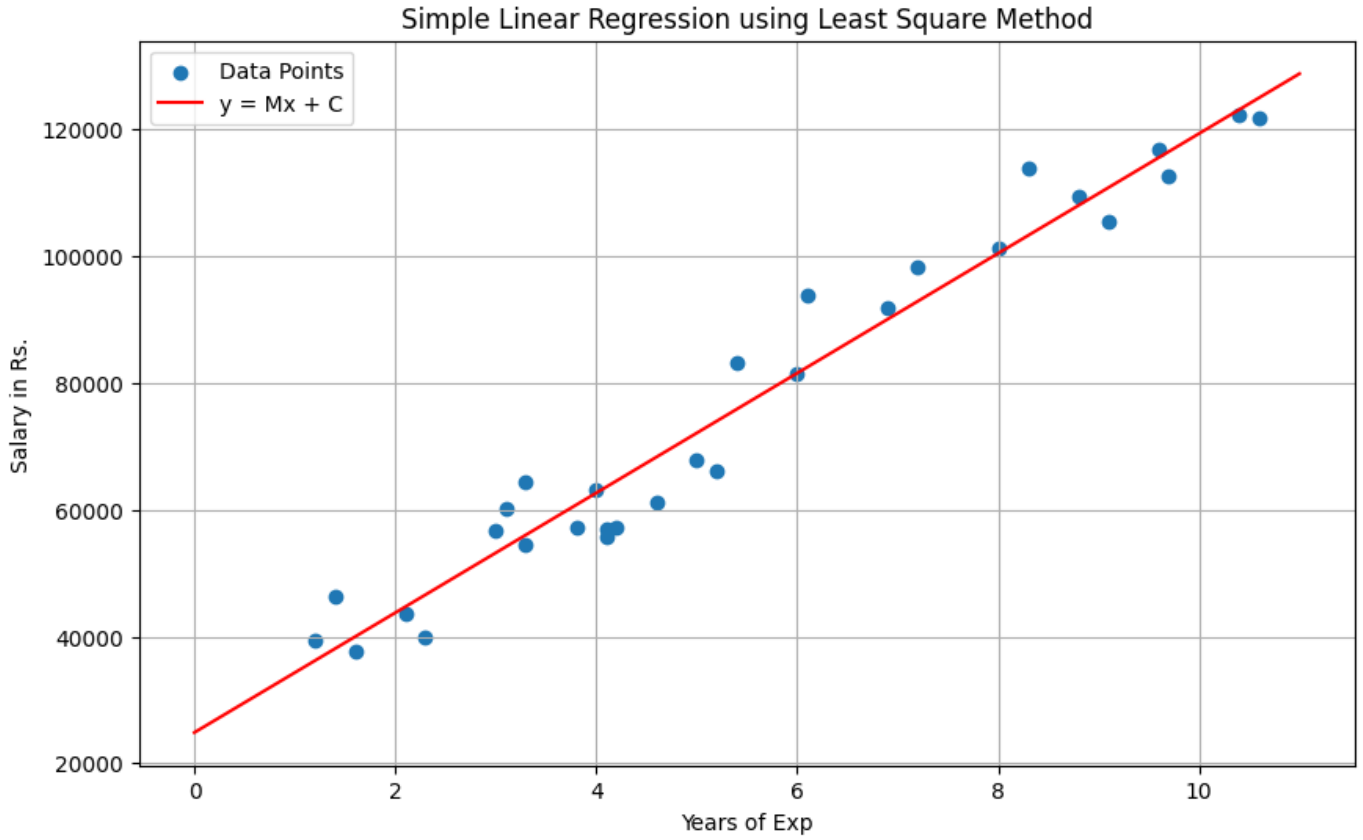
9. Calculate predicted values.
10. Plot the scatter graph and regression line.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

url = 'https://raw.githubusercontent.com/MohamedZaidhA/Machine-
LEarning/refs/heads/main/Salary_dataset.csv'

df = pd.read_csv(url)

df.drop(columns=["Unnamed: 0"], inplace = True)

x="YearsExperience"

y="Salary"

mean_Y = df[y].mean()

mean_X = df[x].mean()

variance_X = df[x].var()

covariance_matrix = df[[x, y]].cov()

covariance = covariance_matrix.loc[x, y]

b = covariance / variance_X

a = mean_Y - b * mean_X

x_values = np.linspace(0, 11, 100)

y_values = b * x_values + a

plt.figure(figsize=(10, 6))

plt.scatter(df[x], df[y], label='Data Points')

plt.plot(x_values, y_values,color="red", label=f'y = Mx + C')

plt.xlabel('Years of Exp')
```

```
plt.ylabel('Salary in Rs.')

plt.grid(True)

plt.legend()

plt.title('Simple Linear Regression using Least Square Method')

plt.show()
```

OUTPUT:



RESULT:

The Simple Linear Regression model was implemented successfully. The regression equation was obtained, and the graph shows a linear relationship between Playing Hours and Performance.

DATASET:

YearsExperience	Salary
1.2	39344
1.4	46206
1.6	37732
2.1	43526
2.3	39892
3	56643
3.1	60151
3.3	54446
3.3	64446
3.8	57190
4	63219
4.1	55795
4.1	56958
4.2	57082
4.6	61112
5	67939
5.2	66030
5.4	83089
6	81364
6.1	93941
6.9	91739
7.2	98274
8	101303
8.3	113813
8.8	109432
9.1	105583
9.6	116970
9.7	112636
10.4	122392
10.6	121873

EXPERIMENT: 3

A PYTHON PROGRAM TO IMPLEMENT LOGISTIC MODEL

Aim:

To implement a Logistic Regression model using Python to predict the probability of Purchasing a product based on study Age and Estimated Salary.

Procedure:

1. Create a dataset Social_Network_Ads.csv containing Purchasing a product, Age and Estimated Salary.
2. Import required Python libraries such as pandas, numpy, and matplotlib.
3. Load the dataset using pandas.
4. Extract Age and Estimated Salary as input variable (x) and purchasing a product as output variable (y).
5. Initialize parameters b_0 (intercept), b_1 (slope1), b2 (slope2) learning rate, and number of epochs.
6. Define the sigmoid function to convert linear output into probability.
7. Compute predicted probabilities using the sigmoid function.
8. Print the final values of b_0 and b_1.
9. Plot the sigmoid curve along with dataset points.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler



url="https://raw.githubusercontent.com/MohamedZaidhA/Machine-
LEarning/refs/heads/main/Social_Network_Ads.csv"
```

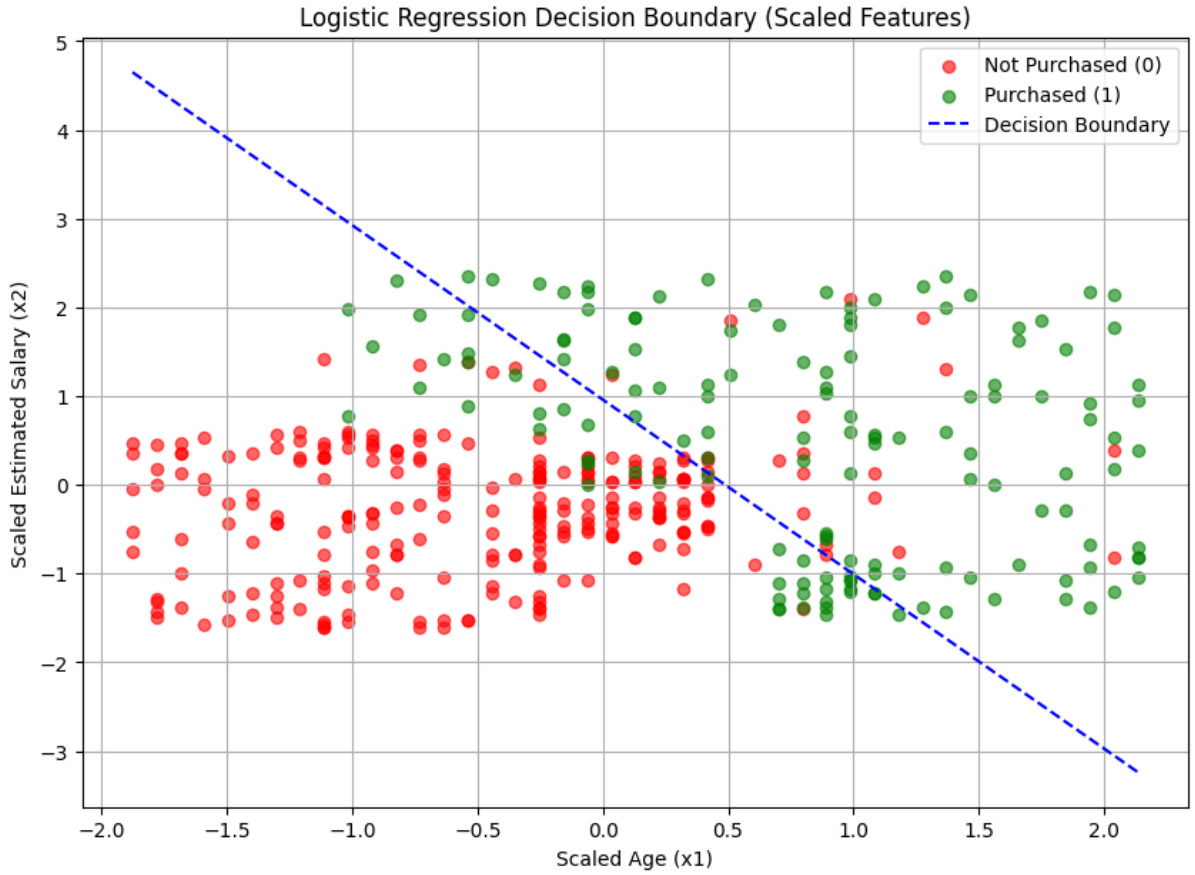
```
data = pd.read_csv(url)


X = data[["Age", "EstimatedSalary"]].values
y = data["Purchased"].values


scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
x1_scaled = X_scaled[:, 0]
x2_scaled = X_scaled[:, 1]


b0 = 0
b1 = 0
b2 = 0
learning_rate = 0.01
epochs = 2000


def sigmoid(z):
    z = np.clip(z, -500, 500)
    return 1 / (1 + np.exp(-z))


for i in range(epochs):
    z = b0 + b1 * x1_scaled + b2 * x2_scaled
    y_pred_prob = sigmoid(z)


    db0 = np.mean(y_pred_prob - y)
    db1 = np.mean((y_pred_prob - y) * x1_scaled)
```

```
    db2 = np.mean((y_pred_prob - y) * x2_scaled)


    b0 -= learning_rate * db0
    b1 -= learning_rate * db1
    b2 -= learning_rate * db2

print("Intercept (b0):", round(b0,4))
print("Slope (b1 for Scaled Age):", round(b1,4))
print("Slope (b2 for Scaled EstimatedSalary):", round(b2,4))

plt.figure(figsize=(10, 7))
plt.scatter(x1_scaled[y == 0], x2_scaled[y == 0], color='red', label='Not Purchased (0)', alpha=0.6)
plt.scatter(x1_scaled[y == 1], x2_scaled[y == 1], color='green', label='Purchased (1)', alpha=0.6)


if abs(b2) > 1e-9:
    x1_boundary = np.array([min(x1_scaled), max(x1_scaled)])
    x2_boundary = (-b0 - b1 * x1_boundary) / b2
    plt.plot(x1_boundary, x2_boundary, color='blue', linestyle='--', label='Decision Boundary')
elif abs(b1) > 1e-9:
    x_intercept = -b0 / b1
    plt.axvline(x=x_intercept, color='blue', linestyle='--', label='Decision Boundary')
else:
    pass
```

```
plt.xlabel("Scaled Age (x1)")

plt.ylabel("Scaled Estimated Salary (x2)")

plt.title("Logistic Regression Decision Boundary (Scaled Features)")

plt.legend()

plt.grid(True)

plt.show()
```

OUTPUT:

```
Intercept (b0): -0.8264
Slope (b1 for Scaled Age): 1.6917
Slope (b2 for Scaled Estimated Salary): 0.8607
```



RESULT:

The Simple Linear Regression model was implemented successfully. The regression equation was obtained.

DATASET:

Age	EstimatedSalary	Purchased
19	19000	0
35	20000	0
26	43000	0
27	57000	0
19	76000	0
27	58000	0
27	84000	0
32	150000	1
25	33000	0
35	65000	0
26	80000	0
26	52000	0
20	86000	0
32	18000	0
18	82000	0
29	80000	0
47	25000	1
45	26000	1
46	28000	1
48	29000	1
45	22000	1
47	49000	1
48	41000	1
45	22000	1
46	23000	1
47	20000	1
49	28000	1
47	30000	1
29	43000	0
31	18000	0
31	74000	0
27	137000	1
21	16000	0
28	44000	0
27	90000	0
35	27000	0
33	28000	0
30	49000	0
26	72000	0
27	31000	0
27	17000	0
33	51000	0
35	108000	0
30	15000	0
28	84000	0

EXPERIMENT: 4

A PYTHON PROGRAM TO IMPLEMENT SINGLE LAYER PERCEPTRON.

Aim:

To implement a Single Layer Perceptron using Python to perform binary classification using the AND gate dataset.

Procedure:

1. Import the NumPy library for numerical operations.

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Where:

- $x_1, x_2, ...x_n$ = input values
- $w_1, w_2, ...w_n$ = weights
- b = bias
- z = weighted sum

2. Define the input dataset representing the AND gate combinations.

3. Define the corresponding target output values.

Activation Function (Step Function)

$$y = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

4. Initialize the weights and bias to zero.

5. Set the learning rate and number of epochs for training.

6. Calculate the linear output using the dot product of inputs and weights.

7. Apply the step activation function to obtain predicted output.

8. Calculate the error between actual output and predicted output.

9. Update weights and bias using the perceptron learning rule.
10. Repeat the process for the specified number of epochs.
11. Print the final weights, bias, and predicted outputs.

CODE:

```
import numpy as np

import pandas as pd

url="https://raw.githubusercontent.com/MohamedZaidhA/Machine-
LEarning/refs/heads/main/Single%20Layer%20Perceptron%20Dataset.csv"

df=pd.read_csv(url)

df = df.dropna()

X = df.iloc[:, 1:-1].values

y = df.iloc[:, -1].values

weights = np.zeros(X.shape[1])

bias = 0

learning_rate = 0.1

epochs = 10

print("Training Perceptron...")

for epoch in range(epochs):

    for i in range(len(X)):

        linear_output = np.dot(X[i], weights) + bias


        if linear_output >= 0:

            y_pred = 1

        else:
```

```
            y_pred = 0

        error = y[i] - y_pred

        if error != 0:
            weights += learning_rate * error * X[i]
            bias += learning_rate * error

print("\nFinal Weights:", weights)
print("Final Bias:", bias.round(3))
print("\nTesting Perceptron:")
correct_predictions = 0
for i in range(len(X)):
    linear_output = np.dot(X[i], weights) + bias
    if linear_output >= 0:
        prediction = 1
    else:
        prediction = 0
    print("Input:", X[i], "Prediction:", prediction)
```

OUTPUT:

```
Training Perceptron...
Final Weights: [ 0.2   -0.274 -0.458]
Final Bias: 0.2
```

```
Testing Perceptron:

Input: [1.   0.08 0.72] Prediction: 1

Input: [1.  0.1 1. ] Prediction: 0

Input: [1.   0.26 0.58] Prediction: 1

Input: [1.   0.35 0.95] Prediction: 0

Input: [1.   0.45 0.15] Prediction: 1

Input: [1.  0.6 0.3] Prediction: 1

Input: [1.   0.7  0.65] Prediction: 0

Input: [1.   0.92 0.45] Prediction: 0

Input: [1.   0.42 0.85] Prediction: 0

Input: [1.   0.65 0.55] Prediction: 0

Input: [1.  0.2 0.3] Prediction: 1

Input: [1.  0.2 1. ] Prediction: 0

Input: [1.   0.85 0.1 ] Prediction: 1
```

RESULT:

The Single Layer Perceptron model was successfully implemented using Python.

DATASET:

Pattern	Feature1	Feature2	Feature3	Class_Label
1	1	0.08	0.72	1
2	1	0.1	1	1
3	1	0.26	0.58	1
4	1	0.35	0.95	0
5	1	0.45	0.15	1
6	1	0.6	0.3	1
7	1	0.7	0.65	0
8	1	0.92	0.45	0
9	1	0.42	0.85	0
10	1	0.65	0.55	0
11	1	0.2	0.3	1
12	1	0.2	1	0
13	1	0.85	0.1	1

EXPERIMENT: 5

A python program to implement multi layer perceptron with back propagation

Aim:

To implement a Multilayer Perceptron (MLP) model using a given dataset and to classify the output based on input features using a neural network.

Procedure:

1. Import required libraries and load the dataset.
2. Split data into features (X) and target (y), then apply train-test split.
3. Normalize data using feature scaling.
4. **Forward Propagation:**
   Compute outputs using:
   - $Z = WX + b$
   - $A = f(Z)$ (activation function like ReLU/Sigmoid)
5. **Backpropagation:**
   - Error = $y - \hat{y}$
   - Update weights:
     $W = W + \eta \cdot \nabla W$
6. Train the MLP model using training data.
7. Predict output for test data.
8. Evaluate using accuracy and confusion matrix.

CODE:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score, confusion_matrix

data = pd.read_csv("MLP.csv")

print("Dataset Preview:")

print(data.head())

X = data.iloc[:, :-1]

y = data.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)



mlp = MLPClassifier(hidden_layer_sizes=(6,6),

                    activation='relu',

                learning_rate_init=0.01,

                    max_iter=500)

mlp.fit(X_train, y_train)

y_pred = mlp.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

plt.figure()

plt.scatter(X_test[:,0], X_test[:,1], c=y_pred)

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")
```

```
plt.title("MLP Predictions")

plt.show()

plt.figure()

plt.plot(mlp.loss_curve_)

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.title("Loss Curve")

plt.show()
```

OUTPUT:

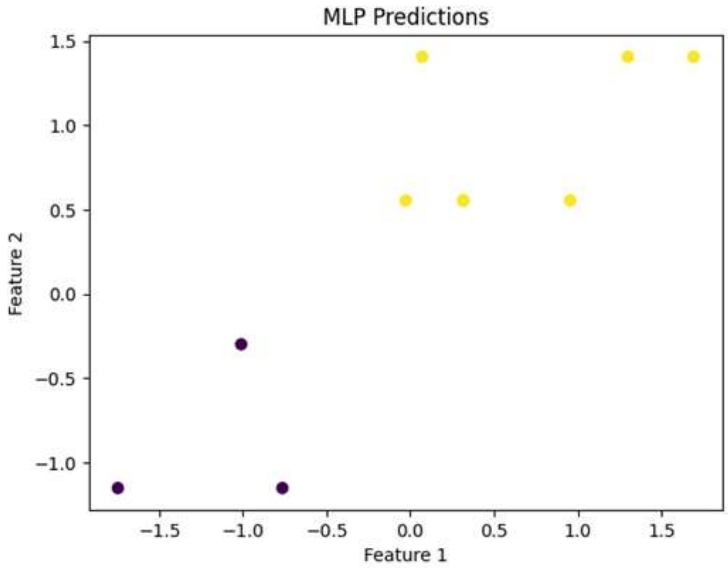
Dataset Preview:

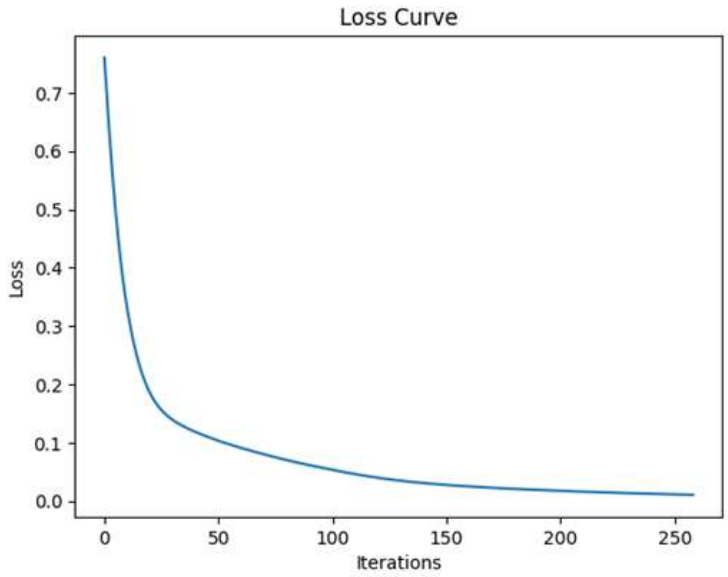
	StudyHours	SleepHours	PracticeTests	ExamResult
0	1.0	5	0	0
1	2.0	6	1	0
2	1.5	5	0	0
3	3.0	6	1	0
4	2.5	6	1	0

Accuracy: 1.0

Confusion Matrix:

```
 [[3 0]
 [0 7]]
```





RESULT:

The Multilayer Perceptron (MLP) model was successfully trained and achieved good accuracy in classifying the data.

EXPERIMENT: 6

A PYTHON PROGRAM TO DO FACE RECOGNITION USING SVM CLASSIFIER.

Aim:

To implement face recognition using Support Vector Machine (SVM) classifier.

Procedure:

1. Import required libraries.
2. Load face dataset (Olivetti dataset).
3. Split dataset into training and testing sets.
4. Train the SVM classifier.
5. Predict test data using the trained model.
6. Evaluate accuracy.

CODE:

```
!pip install scikit-learn
```

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_olivetti_faces

from sklearn.svm import SVC

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score
```

```
faces = fetch_olivetti_faces()

X = faces.data      # flattened images
y = faces.target    # labels (0–39)

X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

model = SVC(kernel='linear')
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

for i in range(5):
   plt.imshow(X_test[i].reshape(64,64), cmap='gray')
   plt.title(f"Predicted: {y_pred[i]}")
   plt.axis('off')
   plt.show()
```

OUTPUT:

Accuracy: 0.975

RESULT:

The SVM classifier successfully recognized faces with good accuracy.

EXPERIMENT: 7

A PYTHON PROGRAM TO IMPLEMENT DECISION TREE

Aim:

To implement a Decision Tree classifier for classification.

Procedure:

1. Import necessary libraries.
2. Load dataset (Iris dataset).
3. Split data into training and testing sets.
4. Train Decision Tree model.
5. Perform prediction on test data.
6. Evaluate accuracy.

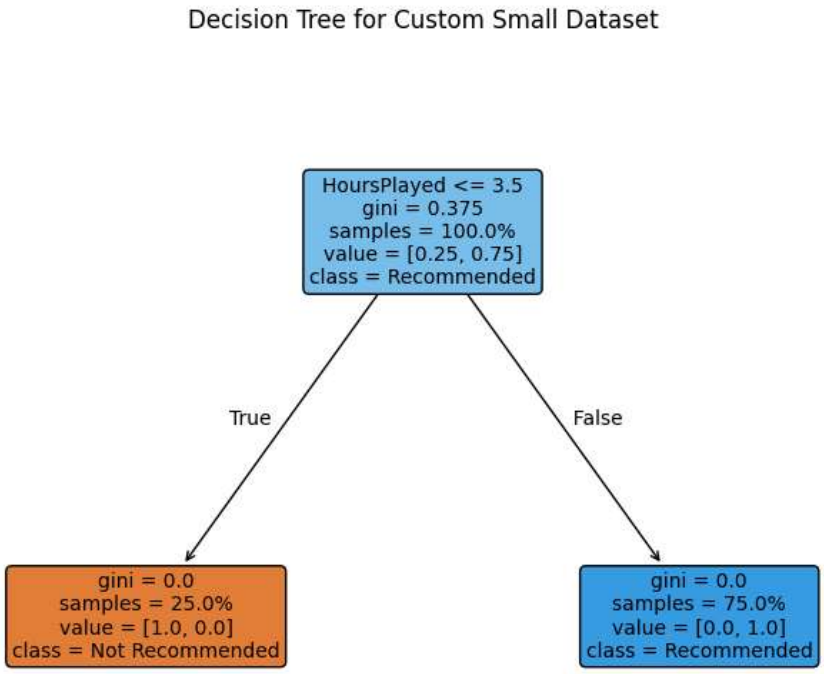
CODE:

```
import pandas as pd

from sklearn.tree import DecisionTreeClassifier, plot_tree

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt



data_custom = {

   'HoursPlayed': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

   'EnjoymentScore': [3, 4, 5, 6, 7, 7, 8, 8, 9, 10],

   'Recommended':    [0, 0, 0, 1, 1, 1, 1, 1, 1, 1]

}
```

```
df_custom = pd.DataFrame(data_custom)

X_custom = df_custom[['HoursPlayed', 'EnjoymentScore']]

y_custom = df_custom['Recommended']

X_custom_train, X_custom_test, y_custom_train, y_custom_test =
train_test_split(

   X_custom, y_custom, test_size=0.2, random_state=42

)

model_custom = DecisionTreeClassifier(random_state=42)

model_custom.fit(X_custom_train, y_custom_train)

plt.figure(figsize=(10, 7))

plot_tree(

    model_custom,

    feature_names=X_custom.columns,

    class_names=['Not Recommended', 'Recommended'],

    filled=True,

    rounded=True,

    proportion=True,

    fontsize=10

)

plt.title('Decision Tree for Custom Small Dataset')

plt.show()
```

```
y_custom_pred = model_custom.predict(X_custom_test)

print("Accuracy for Custom Dataset:",
accuracy_score(y_custom_test, y_custom_pred))
```

OUTPUT:



RESULT:

The Decision Tree model classified the data correctly with satisfactory accuracy.

EXPERIMENT: 8

A PYTHON PROGRAM TO IMPLEMENT BOOSTING

Aim:

To implement Boosting using AdaBoost algorithm.

Procedure:

1. Import required libraries.
2. Load dataset (Breast Cancer dataset).
3. Split dataset into training and testing sets.
4. Train AdaBoost classifier.
5. Predict output for test data.
6. Calculate accuracy.

CODE:

```
from sklearn.datasets import load_wine

from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score


data = load_wine()


X = data.data

y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=42
)

model = AdaBoostClassifier(n_estimators=50)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

OUTPUT:

Accuracy: 0.9444

RESULT:

The AdaBoost model improved classification accuracy by combining multiple weak learners

EXPERIMENT: 9

A PYTHON PROGRAM TO IMPLEMENT KNN AND K-MEANS.

Aim:

To implement K-Nearest Neighbors (KNN) and K-Means clustering.

Procedure:

KNN:

1. Import libraries.
2. Load dataset (Iris dataset).
3. Split into training and testing data.
4. Train KNN classifier.
5. Predict and evaluate accuracy.

K-Means:

1. Import libraries.
2. Load dataset.
3. Apply K-Means clustering.
4. Assign clusters to data points.
5. Visualize clusters.

CODE:

KNN

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score
```

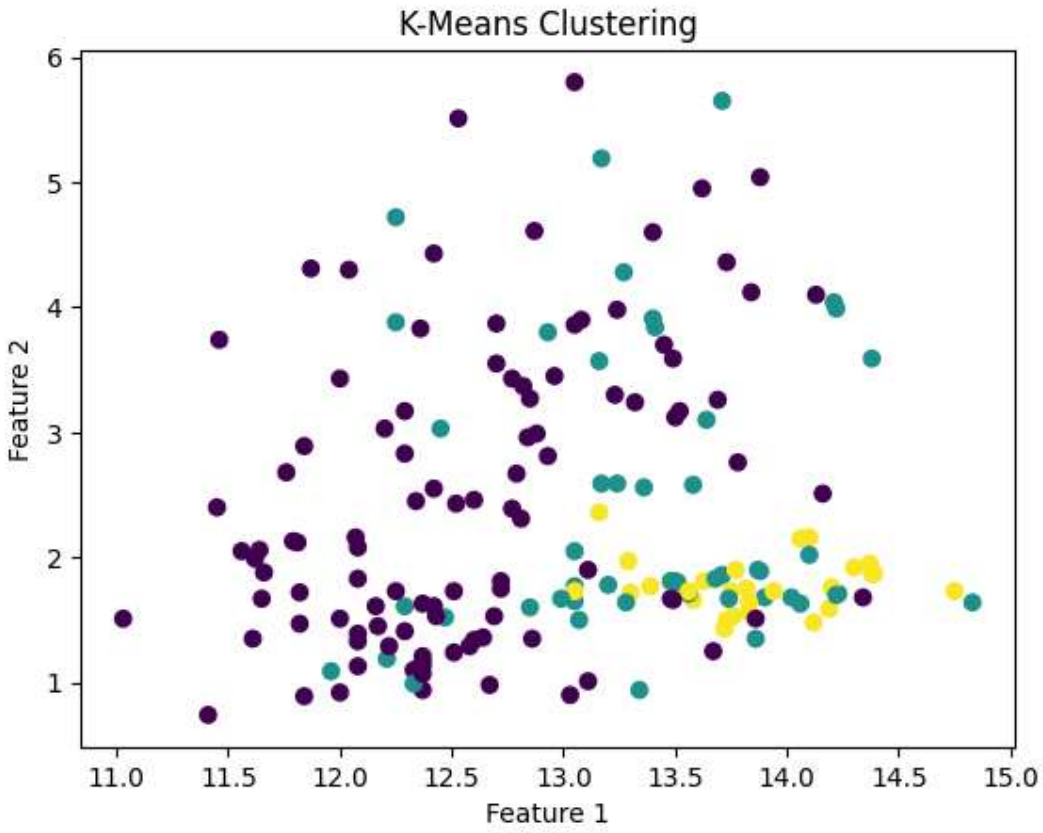
```
data = load_iris()

X = data.data

y = data.target


X_train, X_test, y_train, y_test = train_test_split(

   X, y, test_size=0.2, random_state=42

)


model = KNeighborsClassifier(n_neighbors=3)

model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print("KNN Accuracy:", accuracy_score(y_test,
y_pred))
```

K-Means

```
from sklearn.datasets import load_iris

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt


data = load_iris()


X = data.data

model = KMeans(n_clusters=3, random_state=42)

model.fit(X)


labels = model.labels_

plt.scatter(X[:, 0], X[:, 1], c=labels)

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.title("K-Means Clustering")

plt.show()
```

OUTPUT:

KNN Accuracy: 0.805



RESULT:

KNN successfully classified the data, and K-Means grouped the data into clusters effectively.

EXPERIMENT: 10

A PYTHON PROGRAM TO IMPLEMENT DIMENSIONALITY REDUCTION – PCA

Aim:

To implement dimensionality reduction using Principal Component Analysis (PCA).

Procedure:

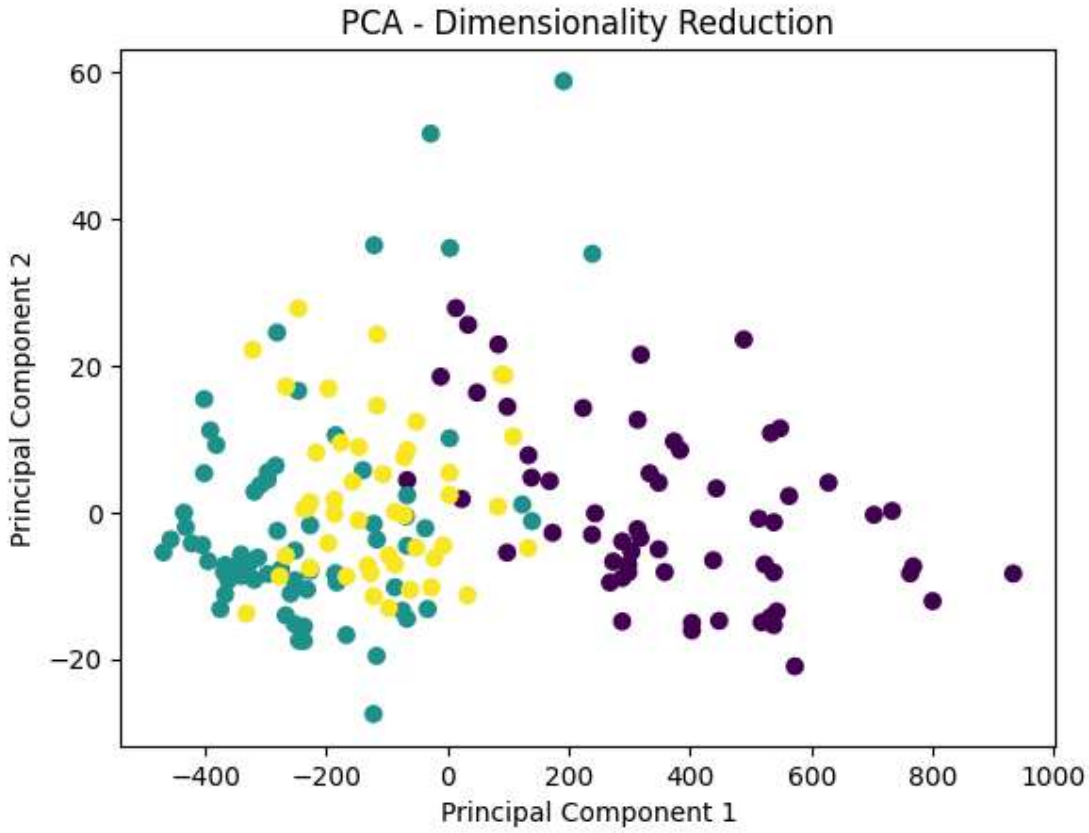
1. Import required libraries.
2. Load dataset (Iris dataset).
3. Apply PCA to reduce dimensions.
4. Transform data into principal components.
5. Visualize reduced data.

CODE:

```
from sklearn.datasets import load_iris

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt

data = load_iris()

X = data.data

y = data.target

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y)
```

```
plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.title("PCA - Dimensionality Reduction")

plt.show()
```

OUTPUT:



KNN Accuracy: 0.8055555555555556

RESULT:

PCA successfully reduced the dataset dimensions while preserving important information.